PORTFOLIO CASE STUDY

Multi-Agent Product Viability System

AI-Orchestrated Business Analysis: From Zero to Executive-Ready Report in an Afternoon

---

3 Hours	$1.51	6 Agents	$15K–$25K
Build Time	Total API Cost	Deployed	Consulting Equivalent

Section 1: What Was Built

A fully functional multi-agent AI system that autonomously evaluates product viability — taking a product concept as input and producing a comprehensive, executive-ready analysis as output. The system uses six specialized AI agents orchestrated through CrewAI, each powered by Claude Sonnet 4.6 via the Anthropic API, running on a local Windows machine with Python 3.12.

Each agent runs in sequence, receiving all prior agents' outputs as context before producing its own. The sixth and final agent — functioning as a VP of Product Strategy — synthesizes all five prior analyses into a single unified investment recommendation document. Output is saved as individual markdown files per agent plus a unified final report.

The system was then stress-tested against a real prototype: the FollowMeWagon autonomous GPS caddy. The analysis it produced was critiqued against executive presentation standards, a structured eight-problem rebuild prompt was developed, a second-generation system was built and verified, and the final report was converted into a polished, self-contained HTML deliverable — complete with embedded prototype photograph, full bill of materials, and V2 upgrade roadmap.

Agent	Role	Key Output
Agent 1	*Market Intelligence Analyst*	Competitive landscape, pricing, market sizing by segment
Agent 2	*Customer Analysis Specialist*	Buyer personas, willingness-to-pay, beachhead recommendation
Agent 3	*Go-to-Market Strategist*	Positioning, marketing copy, launch timeline
Agent 4	*Financial Modeling Analyst*	Per-unit cost at scale, revenue projections, break-even analysis
Agent 5	*Risk Assessment Analyst*	IP, legal, competitive, technical, and financial risk matrix
Agent 6	*VP of Product Strategy*	Unified viability report — synthesizes all prior

(Orchestrator)		outputs into a single investment recommendation

Core Capability

A non-developer with minimal prior Python experience built and deployed a six-agent AI system from scratch in a single afternoon — then evaluated its output with sufficient rigor to identify eight specific quality problems, write a comprehensive rebuild specification, and verify the improved system was functional.

Section 2: Why It Was Built

The project was driven by a direct question: can a complete non-developer build a meaningful, production-relevant multi-agent AI system — without any prior experience in Python, CrewAI, APIs, or agentic frameworks — in a single focused session?

The secondary question was equally important: once the system is built, can you evaluate its output with the same professional rigor you would apply to any high-stakes deliverable — and not just accept AI slop because it looks complete?

Core Question

"Can a non-developer build a functional multi-agent AI system from scratch in an afternoon — and produce output that actually meets the standard a CTO or VP of Product Strategy would accept?"

Section 3: Starting Competence

Prior Python Experience	Effectively zero — no prior Python code written before this project
Prior CrewAI / API Use	None — first time using CrewAI framework or making direct Anthropic API calls
Prior Agentic AI Experience	Minimal — growing familiarity with AI tools from prior portfolio projects under AI guidance
Prior Programming Background	None — no formal or informal software development training
Environment Setup	First time configuring a Python environment, .env files, and npm-style dependency installation on Windows
AI Tool Fluency	Growing — Claude used extensively across prior projects; this was the first time building a system that itself runs AI
Skills Transferred In	Structured planning, disciplined scoping, precision questioning, and the professional instinct to verify before trusting

Section 4: How AI Was Used

System Design & Build

The entire build was conducted in real-time collaboration with Claude. Starting from zero — no Python environment, no familiarity with CrewAI — the session covered environment setup, package installation, agent design, task description writing, context-passing architecture, and execution debugging. Claude functioned as a live technical guide, explaining each step in terms that translated directly into working code.

The resulting crew.py script defined six agents with distinct roles, goals, and backstories; six corresponding tasks with explicit output requirements; and a sequential execution chain that passed each agent's output as context to the next. Total time from first Python install to crew.kickoff() producing seven output files: approximately three hours. Total API cost for the complete build and test run: $1.51.

Key Workflow Insight

The six-agent architecture was not arbitrary. Each agent was designed to represent a distinct professional discipline — market research, customer analysis, go-to-market strategy, financial modeling, risk assessment, and executive synthesis. Passing all prior outputs as context to each subsequent agent allowed the system to reason across disciplines, not just within them. The orchestrator agent was explicitly instructed to identify contradictions between agents and resolve them — not summarize them.

Output Critique & Quality Assessment

After the system ran successfully, the output was subjected to a structured professional critique — evaluated against the standard of a report suitable for presentation to a CTO or VP of Product Strategy. Eight specific quality problems were identified:

- Output truncation — at least three agents cut off mid-sentence due to token limits
- No validation loop — inconsistencies reached the final report uncorrected
- Financial model contained unvalidated assumptions presented as facts
- GTM agent received no technical context — wrote false performance specifications into marketing copy
- Market data had no citations or confidence flags on material figures
- Two required hardware tables were absent entirely
- Synthesis agent produced an incomplete executive document with a cut-off final section
- No completeness check before synthesis — orchestrator worked from potentially truncated inputs

None of these problems were surprising. They were expected — and naming them precisely is itself a demonstration of the capability. Knowing what good looks like, and being willing to document the gap honestly rather than accept polished-looking slop, is the professional standard applied here.

Key Workflow Insight

The rebuild prompt produced from this critique was as valuable as the original build. It specified eight ordered architectural fixes, defined success criteria for each, named exact agent-level changes, and included the known prototype specifications as source-of-truth data for the hardware tables. The second-generation system was built in the other conversation, verified functional, and the improved report confirmed to meet intent.

HTML Report Conversion

The final validated report was converted into a polished, self-contained HTML document — styled as a professional executive deliverable with a dark editorial typography system, confidence-flagged data tables, color-coded priority pills, a V1-vs-V2 comparison panel, a full bill of materials, and an embedded prototype photograph. The HTML file is fully standalone: no external dependencies, print-ready, and mobile-responsive.

Section 5: Timeline & Conditions

Total Build Time	~3 hours — single session, zero to working six-agent system
Total API Cost	$1.51 (Anthropic usage, confirmed via dashboard)
Critique Session	Same session — structured eight-problem quality assessment
Rebuild Prompt	Produced and executed in second conversation; improved system verified functional
HTML Deliverable	Converted from final report markdown with embedded prototype photo
Work Conditions	Primary job and family obligations running concurrently
External Deadline	None — self-directed, self-imposed success criteria
Platform	Python 3.12, CrewAI, Anthropic API, local Windows environment

Section 6: Outcome & Evidence

The delivered system is a functional six-agent multi-agent AI pipeline that takes a product concept as input and autonomously produces a comprehensive product viability analysis — including competitive landscape, customer segmentation, go-to-market strategy, financial model at three production scales, risk assessment, and a unified investment recommendation — in a single execution run at a cost of $1.51.

The comparable cost of commissioning equivalent work from a boutique strategy consulting firm is $15,000–$25,000 for a defined project of this scope, based on 2026 industry benchmarks for competitive analysis and go-to-market strategy engagements (InvoiceBloom, 2026 Consulting Rate Benchmarks). The system compresses that analytical value by a factor of more than 10,000x on cost, and delivers it in minutes rather than weeks.

The output was not accepted at face value. It was stress-tested, critiqued at a professional standard, and a documented rebuild specification was produced and executed. The final HTML report represents the fully-realized deliverable: structured, sourced, complete, and suitable for executive presentation.

Section 7: Key Metrics

Build Time	~3 hours — zero to working six-agent system producing seven output

	files
Total API Cost	$1.51 (Anthropic usage dashboard)
Agents Deployed	6 — running in sequence, each passing output as context to the next
Output Documents	7 — six individual agent files + unified final report
Consulting Equivalent	$15,000–$25,000 per InvoiceBloom 2026 consulting rate benchmarks for equivalent scope
Cost Compression	>10,000x — same analytical depth at a fraction of the cost
Primary AI Platform	Claude Sonnet 4.6 via Anthropic API (all six agents)
Orchestration Framework	CrewAI (Python 3.12, local Windows environment)
Output Format	Markdown files per agent + unified final report + polished HTML deliverable
v2 Rebuild	Completed — eight-problem critique translated into full rebuild prompt; improved system verified functional

Section 8: Lessons Learned

Lesson 1: Multi-Agent Architecture Multiplies Output Quality — If You Design It Right

A single AI agent asked to produce a market analysis, financial model, and risk assessment simultaneously produces mediocre work in all three. Six specialized agents, each focused on a single domain, each receiving the accumulated context of all prior agents, produce dramatically more rigorous output. The architecture is the leverage. Getting the agent roles, task descriptions, and context chain right before writing a single line of code is where the real design work happens.

Lesson 2: Build Time Is Not the Measure — Output Quality Is

The system ran in three hours. That is genuinely impressive and worth stating. But speed without quality is a demo, not a deliverable. The professional standard is not 'it ran' — it is 'would a CTO act on this?' Applying that standard honestly, identifying the gap, documenting it precisely, and rebuilding to close it is the work that actually matters. Anyone can build a system that produces output. The capability demonstrated here is building one that produces correct output — and knowing the difference.

Lesson 3: AI Slop Is a Choice

Every quality problem identified in the first-generation output was predictable. Token truncation, unvalidated financial assumptions, false marketing claims, missing tables — these are not surprises if you know what you're looking at. The professional failure mode is not building a flawed first version. It is accepting the flawed first version because it looks complete. The standard applied throughout this project — refuse to allow any AI slop to make it through — is not a technical requirement. It is a professional one.

Lesson 4: The Critique Is as Valuable as the Build

The rebuild prompt produced from the quality assessment is itself a sophisticated document: eight ordered architectural fixes, agent-level specifications, success criteria, and a source-of-truth data block for the hardware tables. That document did not exist until someone with domain knowledge read the output critically, named what was wrong specifically, and translated the findings into actionable engineering requirements. That translation — from 'this isn't good enough' to 'here is exactly how to fix it' — is a skill that compounds across every project.

What This Project Proves

Zero-to-production agentic AI: A functional six-agent multi-agent system was designed, built, and deployed in three hours with no prior experience in any relevant technical domain.

Executive-standard output: The system produced a comprehensive product viability analysis across five professional disciplines, synthesized into a single investment recommendation — at a fraction of the cost of equivalent human consulting work.

Quality enforcement under pressure: Eight specific quality problems were identified, documented, and resolved through a structured rebuild — not glossed over because the first version looked complete.

Full-stack AI execution: Concept → working system → critique → rebuild → HTML deliverable — a complete execution arc, not a partial proof of concept.

"I can develop fully functional multi-agent systems from scratch in an afternoon. And not only that, but also ensure that they can produce a correct and viable output. I refuse to allow any AI slop to make it through."

--- END OF CASE STUDY ---